

PRD

# Doyle's Stock Chart

실시간 모의 투자 웹 대시보드 — 제품 요구사항 문서

|       |                                     |
|-------|-------------------------------------|
| 문서 유형 | PRD (Product Requirements Document) |
| 버전    | v1.0                                |
| 작성일   | 2026.05.28                          |
| 작성자   | Doyle                               |
| 상태    | Draft                               |

## 목차

|    |                   |
|----|-------------------|
| 01 | 기획 배경 — 왜 만들게 되었나 |
| 02 | 서비스 목표            |
| 03 | 기술 스택             |
| 04 | 핵심 설계 원칙          |
| 05 | 로그인 시스템           |
| 06 | 메인 페이지 & UI 구조    |
| 07 | 기능 명세 — 모달 & 인터랙션 |
| 08 | 실시간 데이터 아키텍처      |
| 09 | 상태 관리 & API 설계    |
| 10 | 데이터베이스 설계         |
| 11 | 성능 최적화            |
| 12 | 기대 효과 & 마무리       |

## 기획 배경 — 왜 만들게 되었나

### 시작점: 흩어진 정보, 느린 확인

주식 투자를 시작하면서 가장 불편했던 점은 단순했다. 내가 보유한 종목의 현재가와 수익률을 한눈에 볼 수 있는 나만의 공간이 없다는 것. 기존 증권사 앱들은 기능이 지나치게 많고 복잡했고, 단순히 '지금 내 주식이 얼마인지'를 확인하는 데도 여러 단계를 거쳐야 했다. 결국 스프레드시트를 만들어 수동으로 관리하거나, 여러 앱을 오가며 가격을 일일이 확인하게 되었다.

이 프로젝트는 그 불편함에서 출발했다. 복잡한 기능 없이, 내가 관심 있는 종목만 등록하고 실시간으로 수익률과 평가금액을 즉시 확인할 수 있는 — 나만을 위한 개인 투자 대시보드를 만들기로 했다.

### 사용자가 겪는 문제 흐름

|   |          |                  |
|---|----------|------------------|
| 1 | 증권 앱 실행  | ■■ ■■ ■■■ ■■     |
| 2 | 종목 검색    | ■ ■■■ ■■ UI/UX   |
| 3 | 수익률 계산   | ■■■■■■ ■■ ■■     |
| 4 | 포트폴리오 확인 | ■■■ ■■, ■■ ■■ ■■ |
| 5 | 시간 낭비    | ■■ ■■■ 5■ ■■ ■■  |

### 해결 방향

|              |                            |
|--------------|----------------------------|
| ✓ 직접 만든다     | 불필요한 기능 없이 핵심만             |
| ✓ 실시간 반영     | 새로고침 없이 현재가 자동 업데이트        |
| ✓ 나만의 종목 관리  | 내가 원하는 종목만 추가/삭제           |
| ✓ 수익률 즉시 확인  | 매수 평균가 기준 수익률 자동 계산        |
| ✓ 금융 앱 수준 UX | Toss / Upbit 느낌의 세련된 인터페이스 |

## 서비스 목표

### 핵심 목표 (North Star)

"내가 보유한 주식을 실시간으로, 빠르게, 한눈에 파악한다."

### 목표 항목

|   |                 |                            |
|---|-----------------|----------------------------|
| ☑ | 보유 종목 실시간 현황 파악 | 1분 이내 가격 갱신, Socket.io 푸시  |
| ☑ | 수익률 자동 계산       | 매수가 기준 수익/손실 즉시 표시         |
| ☑ | 종목별 차트 제공       | 1일~1년 기간 캔들 차트             |
| ☑ | 금융 서비스 수준 UX    | Toss·Upbit·TradingView 스타일 |
| ☑ | 안전한 사용자 관리      | JWT + Refresh Token 인증     |
| ☑ | 모바일 반응형         | 모든 디바이스에서 동일한 경험           |

## Frontend

|                  |               |
|------------------|---------------|
| React            | UI 컴포넌트 기반 개발 |
| Vite             | 빠른 빌드 및 HMR   |
| TailwindCSS      | 유틸리티 기반 스타일링  |
| React Router DOM | 클라이언트 라우팅     |
| Zustand          | 경량 전역 상태 관리   |
| Axios            | HTTP 클라이언트    |
| TanStack Query   | 서버 상태 캐싱      |
| Framer Motion    | 애니메이션         |
| ApexCharts       | 금융 차트         |

## Backend

|              |                |
|--------------|----------------|
| Node.js      | 서버 런타임         |
| Express.js   | REST API 프레임워크 |
| JWT + bcrypt | 인증 및 암호화       |
| Prisma ORM   | 타입 세이프 DB 접근   |
| Socket.io    | 실시간 양방향 통신     |
| node-cron    | 주기적 가격 갱신 스케줄러 |

## Infra / DB

|                        |            |
|------------------------|------------|
| PostgreSQL             | 관계형 데이터베이스 |
| Oracle Cloud Free Tier | 서버 호스팅     |
| Ubuntu + Nginx         | 리버스 프록시    |



## 핵심 설계 원칙

|    |               |                                        |
|----|---------------|----------------------------------------|
| 1  | API 보안        | 프론트에서 직접 주식 API 호출 금지. 모든 데이터는 백엔드 경유. |
| 2  | KEY 관리        | API KEY는 서버에서만 보관. 환경변수로 관리.           |
| 3  | 캐싱 전략         | 서버에서 1분 단위 캐싱 후 클라이언트에 전달.             |
| 4  | 실시간 통신        | Socket.io 기반 서버 푸시로 실시간 가격 반영.         |
| 5  | 반응형 UI        | 모바일(375px)부터 데스크탑(1280px+)까지 완전 대응.    |
| 6  | 금융 UX         | 상승=빨간색, 하락=파란색. 한국 금융 앱 관례 준수.         |
| 7  | 다크모드 기본       | 기본값은 다크 테마. 눈의 피로 최소화.                 |
| 8  | Skeleton UI   | 데이터 로딩 중 Skeleton 컴포넌트로 레이아웃 유지.       |
| 9  | 트랜잭션 기록       | 모든 매수/매도는 transactions 테이블에 영구 저장.     |
| 10 | stock_code 기반 | 종목명이 아닌 stock_code(예: 005930)로 DB 저장.  |

인증 방식

|               |                                  |
|---------------|----------------------------------|
| Access Token  | JWT, 짧은 유효기간, API 요청마다 헤더 포함     |
| Refresh Token | 만료 시 Access Token 재발급, 보안 저장소 관리 |
| bcrypt        | 비밀번호 단방향 해시 저장 (단순 암호화 금지)       |

로그인 성공 애니메이션 (Framer Motion)

|        |                                         |
|--------|-----------------------------------------|
| Step 1 | 화면 중앙에 '어서오세요 도일님' 텍스트 표시               |
| Step 2 | '어서오세요' 텍스트가 서서히 opacity 0으로 페이드 아웃     |
| Step 3 | '도일님' 텍스트만 남아 중앙 유지                     |
| Step 4 | 텍스트가 우측 상단 헤더 영역으로 이동 (layoutAnimation) |
| Step 5 | 헤더에서 '도일님의 주식 현황'으로 텍스트 전환              |

# 메인 페이지 & UI 구조

## 전체 레이아웃 (와이어프레임)

|                                                                       |                                   |
|-----------------------------------------------------------------------|-----------------------------------|
| HEADER — 로고 + 서비스명   닉네임 표시 + 프로필 + 로그아웃                              |                                   |
| SUMMARY CARDS — TOTAL BUY   TOTAL ASSET   TOTAL RETURN   TOTAL PROFIT |                                   |
| STOCK TABLE (초기: 전체 너비) (종목 클릭 시: 45%)                                | STOCK CHART (종목 클릭 시 등장) (55% 너비) |

## Summary Cards 명세

|              |         |                                                          |
|--------------|---------|----------------------------------------------------------|
| TOTAL BUY    | 총 매수 금액 | 보유 종목 매수가 × 수량 합계                                        |
| TOTAL ASSET  | 총 평가 금액 | 현재가 × 보유 수량 합계                                           |
| TOTAL RETURN | 총 수익률   | $((\text{평가금액} - \text{매수금액}) / \text{매수금액}) \times 100$ |
| TOTAL PROFIT | 총 수익 금액 | 평가금액 - 매수금액                                              |

## 보유 종목 테이블 컬럼

|                   |                        |
|-------------------|------------------------|
| NAME              | 종목명                    |
| AVERAGE PRICE     | 평균 매수가                 |
| AMOUNT            | 보유 수량                  |
| CURRENT PRICE     | 현재가 (실시간)              |
| RETURN            | 수익률 (색상: 상승=빨강, 하락=파랑) |
| EVALUATION AMOUNT | 평가 금액                  |
| OPTION            | 수정 / 판매 / 삭제 드롭다운      |



## 기능 명세 — 모달 & 인터랙션

### 추가 모달

제목: 새 종목을 추가합니다

- 종목명 검색 (자동완성 제공)
- 평균 구매가 입력
- 구매 수량 입력
- 저장 시 DB 저장 + UI 즉시 반영 + 실시간 데이터 연결

모든 모달: backdrop blur 적용

### 수정 모달

제목: 종목을 수정합니다

- 평균 구매가 변경
- 구매 수량 변경
- 종목명은 readonly (변경 불가)

모든 모달: backdrop blur 적용

### 삭제 모달

제목: 정말 삭제하시겠습니까?

- 삭제 후 복구 불가 안내 문구
- 취소 / 삭제 버튼

모든 모달: backdrop blur 적용

### 판매 모달

제목: [종목명]을 판매합니다

- 판매 수량 입력
- 현재 보유 수량 힌트 표시
- 수익 계산: (현재가 - 평균단가) × 판매수량
- 판매 후 보유 수량 감소, transactions 기록

모든 모달: backdrop blur 적용

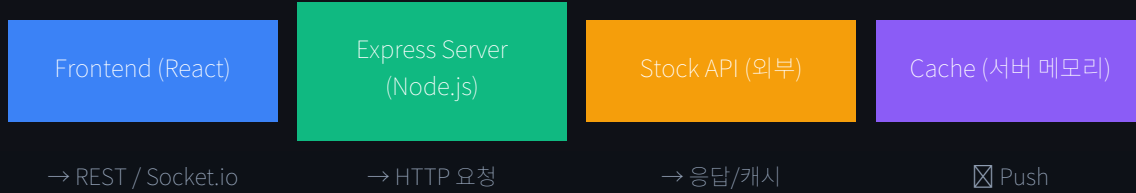
### 차트 패널 명세

|       |                                  |
|-------|----------------------------------|
| 트리거   | 보유 종목 테이블에서 종목 클릭                |
| 위치    | 우측 55% 패널로 슬라이드인 (transition 적용) |
| 표시 정보 | 종목명, 현재가, 등락률, 고가, 저가            |
| 차트 기간 | 1일 / 1주 / 1개월 / 3개월 / 1년         |



## 실시간 데이터 아키텍처

### 데이터 흐름 다이어그램



### Socket.io 실시간 이벤트

|                 |            |                |
|-----------------|------------|----------------|
| price:update    | 서버 → 클라이언트 | 1분 주기 가격 갱신 푸시 |
| portfolio:join  | 클라이언트 → 서버 | 사용자 포트폴리오 룸 참여 |
| portfolio:leave | 클라이언트 → 서버 | 포트폴리오 룸 퇴장     |
| holding:added   | 서버 → 클라이언트 | 새 종목 추가 알림     |
| holding:removed | 서버 → 클라이언트 | 종목 삭제 알림       |

## Zustand Store 구조

|              |                                                       |
|--------------|-------------------------------------------------------|
| authStore    | isLoggedIn, user, accessToken, login(), logout()      |
| holdingStore | holdings[], selectedHolding, setSelected(), refresh() |
| priceStore   | prices[], updatePrice(), connectSocket()              |
| modalStore   | activeModal, openModal(), closeModal()                |
| uiStore      | isChartOpen, setChartOpen(), theme                    |

## REST API 엔드포인트

|        |                |                 |
|--------|----------------|-----------------|
| POST   | /auth/signup   | 회원가입            |
| POST   | /auth/login    | 로그인 → JWT 발급    |
| POST   | /auth/refresh  | Access Token 갱신 |
| GET    | /stocks        | 보유 종목 목록 조회     |
| POST   | /stocks        | 종목 추가           |
| PATCH  | /stocks/:id    | 종목 수정 (평균가, 수량) |
| DELETE | /stocks/:id    | 종목 삭제           |
| POST   | /stocks/sell   | 종목 판매 처리        |
| GET    | /market/price  | 현재가 조회 (캐시)     |
| GET    | /market/chart  | 차트 데이터 (기간별)    |
| GET    | /market/search | 종목 검색 자동완성      |

## 데이터베이스 설계

### users

|            |                  |           |
|------------|------------------|-----------|
| id         | INT / PK         | 기본 키      |
| username   | VARCHAR / UNIQUE | 로그인 아이디   |
| password   | VARCHAR          | bcrypt 해시 |
| nickname   | VARCHAR          | 화면 표시 이름  |
| created_at | TIMESTAMP        | 가입 일시     |

### holdings

|               |           |                   |
|---------------|-----------|-------------------|
| id            | INT / PK  | 기본 키              |
| user_id       | INT / FK  | users.id 참조       |
| stock_code    | VARCHAR   | 종목 코드 (예: 005930) |
| stock_name    | VARCHAR   | 종목명 (표시용)         |
| average_price | DECIMAL   | 평균 매수가            |
| quantity      | INT       | 보유 수량             |
| created_at    | TIMESTAMP | 추가 일시             |
| updated_at    | TIMESTAMP | 수정 일시             |

### transactions

|            |          |                |
|------------|----------|----------------|
| id         | INT / PK | 기본 키           |
| user_id    | INT / FK | users.id 참조    |
| holding_id | INT / FK | holdings.id 참조 |
| type       | ENUM     | BUY / SELL     |
| price      | DECIMAL  | 거래 단가          |
| quantity   | INT      | 거래 수량          |
| profit     | DECIMAL  | 실현 손익 (SELL 시) |



## 성능 최적화

|                   |                                     |
|-------------------|-------------------------------------|
| TanStack Query 캐싱 | 서버 데이터 캐시로 불필요한 네트워크 요청 제거          |
| 불필요한 리렌더링 최소화     | React.memo, useCallback, useMemo 활용 |
| 차트 Lazy Loading   | 종목 선택 시에만 ApexCharts 로드             |
| WebSocket 최적화     | 연결 재사용, heartbeat, 자동 재연결 처리        |
| Skeleton UI       | 로딩 중 레이아웃 유지, CLS(누적 레이아웃 이동) 최소화   |
| 이미지/번들 최적화        | Vite 코드 스플리팅, 동적 import             |
| 서버 캐싱             | 1분 단위 가격 캐시, Redis 도입 고려            |

## 반응형 브레이크포인트

|                   |                              |
|-------------------|------------------------------|
| Desktop (1280px+) | 테이블 + 차트 좌우 분할 레이아웃          |
| Tablet (768px~)   | 세로 비율 증가, 차트 하단 배치           |
| Mobile (~767px)   | 카드형 UI, 차트는 하단 Drawer로 슬라이드업 |

## 기대 효과 & 마무리

### 이 서비스를 완성했을 때 기대되는 것들



#### 즉각적인 포트폴리오 확인

앱을 열면 5초 이내에 전체 보유 종목의 현재가와 수익률을 확인할 수 있다. 더 이상 여러 앱을 오가거나 스프레드시트를 업데이트할 필요가 없다.



#### 데이터 기반 투자 판단

종목별 차트와 수익률 추이를 한 화면에서 비교함으로써, 감에 의존한 투자가 아닌 데이터 기반의 매도/매수 결정을 내릴 수 있다.



#### 실시간 가격 변동 인식

Socket.io 기반 푸시 방식으로 새로고침 없이 가격이 자동 갱신된다. 급격한 가격 변동에도 빠르게 대응할 수 있다.



#### 포트폴리오 역량 증명

단순 CRUD를 넘어 실시간 통신, JWT 인증, 금융 데이터 처리, 고품질 UI/UX를 갖춘 풀스택 프로젝트로 개발 역량을 입증한다.



#### 확장 가능한 아키텍처



현재는 개인 용도이지만, 사용자 추가와 알림 기능, AI 기반 투자 추천 등으로 확장할 수 있는 구조를 처음부터 갖춘다.



#### 개발과 디자인의 통합

Toss·Upbit·TradingView를 참고한 고급 금융 대시보드 디자인으로 '개발만 하는 개발자'가 아닌, UX를 이해하는 개발자임을 보여준다.



이 프로젝트는 불편함에서 시작되었다.  
그 불편함을 해소하는 과정에서 진짜 실력이 만들어진다.

Doyle's Stock Chart — 나만을 위한 실시간 투자 대시보드

PRD v1.0 · Doyle's Stock Chart · 작성일: 2026.05.28